

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

10. August 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Author) . . . 1
1.2 Zielsetzung der Arbeit (Author) . . . 1
1.3 Methodik (Author) . . . 1

2 Übergreifende Architektur & Systemdesign **1**
2.1 Technologiestack (Author) . . . 1
2.2 Modulare Softwarearchitektur (Author) . . . 1
2.3 Unified UI-Konzept (Author) . . . 1
2.4 Simulator-Architektur (Author) . . . 1
2.5 Datenmodellierung (Author) . . . 1

3 Einsatz von Künstlicher Intelligenz im Projekt **1**
3.1 KI Reflexion (Markus Nöth) . . . 1
3.1.1 KI als Co-Developer (Markus Nöth) . . . 2
3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth) . . . 2
3.1.3 Kritische Reflexion (Markus Nöth) . . . 2
3.2 KI Reflexion (Richard Schröder) . . . 2
3.2.1 KI als Co-Developer (Richard Schröder) . . . 2
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 2
3.2.3 Kritische Reflexion (Richard Schröder) . . . 2
3.3 KI Reflexion (Enrico Ebert) . . . 2
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 2
3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert) . . . 2
3.3.3 Kritische Reflexion (Enrico Ebert) . . . 2
3.4 KI Reflexion (Duy Tan Pham) . . . 2
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 2
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 3
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 3
3.5 KI Reflexion (Kristian Popov) . . . 3
3.5.1 KI als Co-Developer (Kristian Popov) . . . 3
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 4
3.5.3 Kritische Reflexion (Kristian Popov) . . . 4

4 Designentscheidungen der Fachmodule **5**
4.1 Turing Maschine (Author) . . . 5
4.1.1 Entscheidungen in der Architektur (Author) . . . 5
4.1.2 Entscheidungen im UI (Author) . . . 5
4.1.3 Andere Entscheidungen (Author) . . . 5
4.2 Endliche Automaten (Kristian Popov) . . . 5
4.2.1 Entscheidungen in der Architektur (Kristian Popov) . . . 5
4.2.2 Entscheidungen im UI (Kristian Popov) . . . 6
4.2.3 Andere Entscheidungen (Kristian Popov) . . . 7
4.3 Reguläre Sprachen (Kristian Popov) . . . 8
4.3.1 Entscheidungen in der Architektur (Kristian Popov) . . . 8
4.3.2 Entscheidungen im UI (Kristian Popov) . . . 10
4.3.3 Andere Entscheidungen (Kristian Popov) . . . 10
4.4 Zahlensysteme (Author) . . . 11
4.4.1 Entscheidungen in der Architektur (Author) . . . 11

4.4.2 Entscheidungen im UI (Author) . . . 11
4.4.3 Andere Entscheidungen (Author) . . . 11
4.5 Moore & Mealy Automaten (Author) . . . 11
4.5.1 Entscheidungen in der Architektur (Author) . . . 11
4.5.2 Entscheidungen im UI (Author) . . . 11
4.5.3 Andere Entscheidungen (Author) . . . 11
4.6 Disjunktive Normalform (Author) . . . 11
4.6.1 Entscheidungen in der Architektur (Author) . . . 11
4.6.2 Entscheidungen im UI (Author) . . . 11
4.6.3 Andere Entscheidungen (Author) . . . 11
4.7 Encoding (Author) . . . 11
4.7.1 Entscheidungen in der Architektur (Author) . . . 11
4.7.2 Entscheidungen im UI (Author) . . . 11
4.7.3 Andere Entscheidungen (Author) . . . 11
4.8 Java (Author) . . . 11
4.8.1 Entscheidungen in der Architektur (Author) . . . 11
4.8.2 Entscheidungen im UI (Author) . . . 11
4.8.3 Andere Entscheidungen (Author) . . . 11

5 Evaluation & Projekterfahrung 12
5.1 Qualitätssicherung (Author) . . . 12
5.2 Abgleich mit dem Lastenheft (Author) . . . 12
5.3 Lessons Learned (Author) . . . 12
5.4 Design-Iterationen (Author) . . . 12

6 Fazit & Ausblick 12
6.1 Zusammenfassung der Projektergebnisse (Author) . . . 12
6.2 Perspektiven (Author) . . . 12

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . i

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Eine Arbeit von Technische Hochschule Würzburg-Schweinfurt (THWS) Studenten.

1.1 Problemstellung (Author)

Die Hürde der Abstraktion in Grundlagen Informatik

1.2 Zielsetzung der Arbeit (Author)

Didaktische Aufbereitung komplexer Modelle durch interaktive Simulationen

1.3 Methodik (Author)

Vorgehensweise im Team und Rollenverteilung

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Author)

React 19, TypeScript, Vite, Tailwind CSS 4, shadcn/ui, Zustand, i18next, Lucide Icons

2.2 Modulare Softwarearchitektur (Author)

Single Source of Truth (`moduleCatalog.tsx`), lazy Loading, Shared Topic Pattern – Sicherstellung der Erweiterbarkeit für neue Themen

2.3 Unified UI-Konzept (Author)

Wiederverwendbare Shared Components, konsistentes Designsystem (CSS-Variablen, Dark Mode, moduleigene Akzentfarben, Geist Font)

2.4 Simulator-Architektur (Author)

Das `ISimulator<TConfig, TState>`-Interface und der generische `useSimulationEngine`-Hook als Grundlage aller schrittweisen Simulationen

2.5 Datenmodellierung (Author)

Import/Export über `ISimulatorTransferCodec` mit Base64-Kodierung für Vorlesungskompatibilität

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI Reflexion (Markus Nöth)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.1.1 KI als Co-Developer (Markus Nöth)

Reflektieren und so...

3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth)

Reflektieren und so...

3.1.3 Kritische Reflexion (Markus Nöth)

Reflektieren und so...

3.2 KI Reflexion (Richard Schröder)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.2.1 KI als Co-Developer (Richard Schröder)

Reflektieren und so...

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Reflektieren und so...

3.2.3 Kritische Reflexion (Richard Schröder)

Reflektieren und so...

3.3 KI Reflexion (Enrico Ebert)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.3.1 KI als Co-Developer (Enrico Ebert)

Reflektieren und so...

3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert)

Reflektieren und so...

3.3.3 Kritische Reflexion (Enrico Ebert)

Reflektieren und so...

3.4 KI Reflexion (Duy Tan Pham)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.4.1 KI als Co-Developer (Duy Tan Pham)

Reflektieren und so...

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Reflektieren und so...

3.4.3 Kritische Reflexion (Duy Tan Pham)

Reflektieren und so...

3.5 KI Reflexion (Kristian Popov)

Mein Schwerpunkt lag während des Projekts auf den Modulen *Endliche Automaten* und *Reguläre Sprachen*. Ich habe dabei mehrere Modelle ausprobiert: Gemini 3.1 Pro, die kostenlose Version von ChatGPT sowie Claude Opus 4.8 und später Opus 5.

Jedes Modell hatte seine eigenen Stärken. Manche konnten theoretische Zusammenhänge besser erklären, andere waren stärker bei der Implementierung oder bei der Fehlersuche. Überwiegend habe ich mit Claude gearbeitet, weil das Modell bei Programmieraufgaben seltener fehlerhafte oder frei erfundene Vorschläge lieferte und ich die Antworten dadurch weniger oft nacharbeiten musste.

Erst mit der Zeit ist mir aufgefallen, dass die Modelle unterschiedlich viel Aufwand in die Erstellung einer Antwort investieren. Viele Modelle planen vor der eigentlichen Antwort mehr oder weniger ausführlich; der Planungsaufwand lässt sich meist in mehreren Stufen einstellen. Eine höhere Stufe bedeutet mehr verbrauchte Tokens, längere Wartezeit und höhere Kosten, führt aber nicht automatisch zu einem besseren Ergebnis. Für kleine, klar umrissene Änderungen war die niedrige Stufe ausreichend und zudem schneller. Erst wenn eine Aufgabe mehrere Dateien betraf oder eine Abwägung zwischen mehreren Lösungswegen verlangte, lieferte die höhere Stufe brauchbare Vorschläge. Deshalb habe ich je nach Aufgabe bewusst entschieden, welche Einstellung sinnvoll war.

3.5.1 KI als Co-Developer (Kristian Popov)

Die KI habe ich nicht über eine Chat-Oberfläche bedient, sondern direkt in die Entwicklungsumgebung eingebunden. Dadurch musste ich Code nicht mehr zwischen Editor und Chat kopieren. Stattdessen konnte das Modell auf die Projektdateien zugreifen, Änderungen vorschlagen und diese direkt im Projekt prüfen.

Besonders hilfreich war die KI bei wiederkehrenden Arbeiten: bei Refactorings und Optimierungen am Quellcode, beim Erzeugen von Boilerplate-Code, beim Schreiben von Unit-Tests sowie bei der Internationalisierung der Oberfläche. Gerade bei den Tests sparte sie viel Zeit. Für die Automatenlogik, das Parsen der Grammatiken und die Validierung der Programme konnte die KI aus der vorhandenen Implementierung schnell passende Testfälle ableiten, auch für Eingaben, an die ich selbst zunächst nicht gedacht hatte.

Außerdem half die KI beim Aufräumen des Projekts, etwa beim Beheben von TypeScript-Warnungen, beim Vereinheitlichen von Benennungen sowie beim Erstellen von Kommentaren und Methodendokumentationen. Mit den neueren Modellen und präzisen Prompts waren viele der vorgeschlagenen Änderungen bereits so gut, dass ich sie direkt übernehmen konnte. Trotzdem habe ich jede Änderung selbst überprüft und getestet.

Bei Architektur- und Implementierungsfragen habe ich die KI bewusst nicht selbst entscheiden lassen. Stattdessen ließ ich mir zunächst mehrere Lösungsansätze mit ihren Vor- und Nachteilen erklären und traf die Entscheidung anschließend selbst. Dieser Ansatz erforderte zwar etwas

mehr Zeit, führte aber zu besser begründeten Architekturentscheidungen und verhinderte, dass ich Vorschläge der KI ungeprüft übernahm.

Möglich wurde diese Arbeitsweise durch mehrere Erweiterungen aus dem offiziellen Plugin-Verzeichnis `claude-plugins-official`. Für meine Module waren vor allem zwei davon relevant:

`typescript-lsp`
: bindet den Sprachserver von TypeScript über das Language Server Protocol (LSP) ein. Der KI stehen dadurch dieselben Typinformationen und Referenzen zur Verfügung wie der Entwicklungsumgebung. Bezeichner und Signaturen musste das Modell also nicht aus dem Quelltext erschließen, sondern konnte gezielt danach suchen, wodurch Änderungen über Dateigrenzen hinweg konsistent blieben.

`chrome-devtools-mcp`
: verbindet die KI über das Model Context Protocol (MCP) mit den Entwicklerwerkzeugen des Browsers. Konsolenausgaben und das Verhalten der laufenden Anwendung ließen sich damit direkt auswerten. Bei der Fehlersuche in den Simulatoren war das der größte praktische Gewinn: Ein fehlerhafter Zustandsübergang wurde am tatsächlichen Ablauf sichtbar und musste nicht aus dem Quelltext rekonstruiert werden.

Zusätzlich habe ich das Projekt einmalig mit Graphify [1] analysiert und daraus einen Wissensgraphen erzeugt. Graphify verbindet eine statische Analyse auf Basis von Tree-sitter mit einer semantischen Auswertung durch ein Sprachmodell. Die Knoten des Graphen stehen für Dateien, Klassen und Funktionen, die Kanten für deren Beziehungen untereinander. Zu Beginn einer Sitzung musste die KI das Projekt nicht erneut vollständig einlesen, sondern konnte den für die jeweilige Aufgabe relevanten Teilgraphen abfragen. Gerade bei Änderungen, die mehrere Dateien betrafen, blieb der notwendige Kontext dadurch besser erhalten.

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Für beide Module musste jede Übung mehr enthalten als nur die eigentliche Aufgabenstellung. Zusätzlich wurden eine Referenzlösung, ein Startprogramm sowie Beispielwörter benötigt, mit denen die Lösung automatisch überprüft werden konnte. Erst diese Kombination ergibt ein Lernangebot, das ohne Betreuung funktioniert.

Die KI habe ich hier vor allem zur Variantenbildung eingesetzt. Ausgehend von einer selbst entworfenen Referenzaufgabe ließen sich weitere Aufgaben desselben Typs erzeugen, mit anderem Alphabet, anderer Wortlänge oder anderem Schwierigkeitsgrad. Genau dafür war die KI gut geeignet: Die Struktur steht fest, variiert wird nur der Inhalt.

Ob die Aufgaben tatsächlich korrekt waren, habe ich aber nicht der KI überlassen, sondern im Simulator geprüft. Diese Prüfung läuft deterministisch ab: Geprüft werden alle Wörter über dem gemeinsamen Alphabet bis zur Länge vier, und für jedes dieser Wörter muss der Automat der Studierenden dieselbe Entscheidung treffen wie die Referenzlösung. Bewertet wird damit das tatsächliche Verhalten des Automaten und nicht die Einschätzung eines Sprachmodells. Aus demselben Grund habe ich jede generierte Aufgabe an der Simulation gegengeprüft, bevor sie in den Aufgabenkatalog aufgenommen wurde.

3.5.3 Kritische Reflexion (Kristian Popov)

Die Vorschläge der KI waren oft brauchbar, aber nicht immer richtig oder passend für die bestehende Projektstruktur. Wiederholt schlug die KI APIs vor, die in der verwendeten Version so nicht mehr existierten. Gelegentlich erfand sie auch Funktionen oder Bibliotheken, die es in dieser Form nie gegeben hatte. Gerade Sonderfälle wurden häufig übersehen, und einzelne

Änderungen wirkten sich an Stellen aus, die im Vorschlag selbst gar nicht erwähnt waren. Besonders bei formalen Sprachen reicht es nicht aus, dass eine Lösung plausibel aussieht. Ob ein Automat die geforderte Sprache tatsächlich erkennt, zeigt sich erst beim Durchspielen konkreter Eingaben.

Ein weiteres Problem lag in der Struktur des Codes. Da sich *Endliche Automaten* und *Reguläre Sprachen* stark ähneln, duplizierte die KI bestehende Implementierungen, statt sie zu abstrahieren. So entstanden Simulator- und Übersichtsseiten, die sich zwischen den Modulen kaum unterschieden, und Funktionen, die nie in der Oberfläche eingebunden wurden und später wieder entfernt werden mussten. Betraf eine Änderung beide Module zugleich, etwa die einheitliche Gruppierung von Beispielen und Übungen nach Kategorien, reichte das Kontextfenster des Modells zudem nicht aus, um alle betroffenen Stellen zu erfassen. Solche Aufgaben musste ich in kleinere Schritte zerlegen.

Insgesamt hat mir die KI vor allem bei wiederkehrenden Aufgaben und beim Ausbau bereits vorhandener Funktionen viel Zeit gespart. Gerade bei theoretischen Konzepten und bei Änderungen an mehreren Modulen gleichzeitig war die erste Lösung aber selten schon die richtige. Den Wechsel zwischen den Modellen könnte künftig ein Orchestrator wie Sakana Fugu übernehmen, der je nach Aufgabe automatisch das passende Modell auswählt [2]. Das System erschien allerdings erst kurz vor Abschluss meiner Arbeit, sodass ich es nicht mehr einsetzen konnte. Für mich blieb die KI während des gesamten Projekts ein Werkzeug zur Unterstützung. Die fachlichen Entscheidungen und die Verantwortung für die Korrektheit der Implementierung lagen jedoch jederzeit bei mir.

4 Designentscheidungen der Fachmodule

4.1 Turing Maschine (Author)

4.1.1 Entscheidungen in der Architektur (Author)

4.1.2 Entscheidungen im UI (Author)

4.1.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.2 Endliche Automaten (Kristian Popov)

Im Modul *Endliche Automaten* können Studierende deterministische und nichtdeterministische endliche Automaten selbst definieren und ein Eingabewort Schritt für Schritt ausführen. Jeder Schritt erscheint gleichzeitig in der Übergangstabelle und im Zustandsdiagramm. Die grafische Sicht macht dabei sichtbar, was aus der Tabelle allein nur mit Mühe hervorgeht: Zustände, Übergänge und den Weg, den das Wort durch den Automaten nimmt. Ergänzt wird die Einzelsimulation unter anderem durch einen Stapeltest für mehrere Wörter, die Umwandlung rechtslinearer Grammatiken in Automaten und Übungen mit automatischer Rückmeldung.

4.2.1 Entscheidungen in der Architektur (Kristian Popov)

Ursprünglich sollte das Modul nur deterministische endliche Automaten unterstützen. Die Unterstützung für nichtdeterministische Automaten kam erst später hinzu. Ein Symbol darf in mehrere Zustände zugleich führen, wodurch sich ein Automat oft unmittelbar aus der Beschreibung der Sprache ablesen lässt und zudem deutlich kompakter ausfällt. Der Schritt

zum DFA, bei dem jede Mehrdeutigkeit aufgelöst werden muss, wird damit erst im Anschluss notwendig.

Für die Ausführung ergibt sich daraus eine einfache Konsequenz: Jeder DFA ist zugleich ein NFA. Zwei getrennte Ausführungspfade wären deshalb überflüssig gewesen. Der Kern arbeitet deshalb stets mit einer Menge aktiver Zustände; beim DFA enthält diese Menge höchstens ein Element. Der deklarierte Typ steuert nur die Validierung, nicht die Ausführung.

Der Simulator ist in fünf Verzeichnisse getrennt, die jeweils eine klar umrissene Aufgabe haben:

`domain`
: enthält die Fachlogik: den Datentyp des Automaten, die Schrittfunktion samt ε-Hülle, die Validierung, die Prüfung auf Determinismusverstöße, die Umwandlung aus einer Grammatik, den Stapellauf, die Auswertung der Übungen sowie die Berechnung des Diagrammlayouts.

`application`
: verbindet diese Logik mit React: ein Hook für den Ablauf der Simulation und ein Hook für den Editor, der die Eingaben über einen Reducer verwaltet.

`adapters`
: bereitet den Automaten für die Tabellendarstellung auf, also Kopfzeile, Zeilen je Symbol und die jeweils hervorzuhebende Zelle.

`presentation`
: enthält die Komponenten: Zustandsdiagramm, Übergangstabelle, Eingabewort, Statuskarte, Stapeltest und Grammatikeingabe.

`config`
: hält die Standardautomaten und die Beispiele.

Entscheidend an dieser Aufteilung ist, dass `domain` keine Abhängigkeit zu React besitzt. Die Automatenlogik bleibt dadurch ohne gerenderte Oberfläche testbar, und dieselben Funktionen konnten später im Modul *Reguläre Sprachen* unverändert wiederverwendet werden.

Ein Simulationsschritt verändert den bisherigen Laufzustand nicht, sondern erzeugt einen neuen. Dieser hält die Historie der aktiven Zustände und der genommenen Übergänge fest. Dadurch sind sowohl ein Schritt zurück als auch der Sprung zu einem früheren Schritt möglich, ohne dass die Oberfläche eine zweite, eigene Historie führen muss.

Die Validierung liefert keine fertigen Texte, sondern Fehlercodes und die dazugehörigen Werte, etwa den betroffenen Zustand und das Symbol. Erst die Oberfläche übersetzt diese Informationen in Meldungen. So bleibt die Fachlogik von der Oberflächensprache unabhängig, und dieselben Fehler lassen sich konsistent mehrsprachig ausgeben.

Alle mitgelieferten Automaten sind partiell definiert: Fehlt für ein Symbol ein Übergang, wird das Wort abgelehnt. Auf einen zusätzlich gezeichneten Fangzustand, der nur noch sich selbst erreicht, wurde bewusst verzichtet. Er verändert die erkannte Sprache nicht, vergrößert aber das Diagramm und lenkt von der eigentlichen Struktur ab. Ein automatisierter Test stellt sicher, dass weder ein Beispiel noch eine Referenzlösung einen solchen Zustand enthält.

4.2.2 Entscheidungen im UI (Kristian Popov)

Ursprünglich war für die Simulation nur die Übergangstabelle zusammen mit dem Eingabewort vorgesehen. Formal genügt das: Die Tabelle definiert den Automaten vollständig, das Eingabewort

zeigt die aktuelle Leseposition. Während der Entwicklung zeigte sich jedoch, dass sich der Ablauf so nur schwer verfolgen lässt. Welchen Weg ein Wort nimmt, ist aus der Tabelle nicht unmittelbar ersichtlich; Zyklen und Sackgassen werden erst sichtbar, wenn man mehrere Zeilen gedanklich miteinander verknüpft. Deshalb ergänzt ein Zustandsdiagramm die Tabelle.

Beide Sichten sind an denselben Simulationsschritt gebunden. Die Tabelle hebt genau die Zelle hervor, die dem im Diagramm markierten Übergang entspricht. Damit bleiben beide Darstellungen synchron: Die Tabelle liefert die formale Definition, das Diagramm die anschauliche Darstellung, und ihre Verbindung wird unmittelbar sichtbar.

Die Anordnung der Zustände wird aus der Größe des Automaten abgeleitet. Kleine Automaten werden auf einem Kreis angeordnet, größere über ein kräftebasiertes Verfahren verteilt, bei dem sich Zustände gegenseitig abstoßen und Übergänge sie zusammenziehen. Die Startpositionen werden aus dem Zustandsnamen berechnet und sind damit reproduzierbar. Ohne diese feste Streuung können symmetrische Automaten in einer ungeeigneten Lage zusammenfallen; zudem würde derselbe Automat bei jedem Aufruf anders aussehen. Das Diagramm lässt sich zoomen und verschieben, einzelne Zustände auch frei ziehen. Unterhalb einer Zoomstufe werden Beschriftungen ausgeblendet; aktive und fehlerhafte Übergänge behalten ihre Beschriftung jedoch immer, weil gerade sie in diesem Moment die relevante Information tragen.

Verstöße gegen den Determinismus werden nicht nur als Liste ausgegeben, sondern unmittelbar im Diagramm markiert. Damit ist die Ursache dort sichtbar, wo sie entsteht.

Der Editor umfasst alle Bestandteile der Definition eines Automaten: die Wahl zwischen DFA und NFA, das Eingabealphabet, genau einen Startzustand, beliebig viele Endzustände sowie das Eingabewort für den anschließenden Lauf. Die Zustandsmenge wird nicht als Text gepflegt, sondern direkt in der Übergangstabelle über Schaltflächen erweitert und verkleinert. Die Tabelle dient damit zugleich als Eingabemaske und als Anzeige. Alphabetsymbole sind auf jeweils ein Zeichen beschränkt, damit eine Eingabe wie `10` nicht versehentlich als ein einzelnes Symbol aufgenommen wird. Die Zeile für ε-Übergänge erscheint erst, wenn sie tatsächlich benötigt oder ausdrücklich angefordert wird; für einen DFA wäre sie nur eine Fehlerquelle.

4.2.3 Andere Entscheidungen (Kristian Popov)

Neben dem Einzelschritt gibt es einen Schrittverlauf, über den sich jeder bereits erreichte Zustand direkt auswählen lässt. Der gewünschte Schritt wird dabei vom Anfang des Laufs an neu berechnet, statt ein separates Ergebnis zwischenzuspeichern. Bei den hier üblichen Wortlängen ist das unkritisch und stellt sicher, dass Anzeige und tatsächlicher Ablauf immer übereinstimmen.

Der Stapeltest ergänzt die schrittweise Simulation um den umgekehrten Blick. Statt ein einzelnes Wort genau zu verfolgen, werden mehrere Wörter auf einmal geprüft und mit ihrem Ergebnis aufgelistet. So lässt sich besser einschätzen, ob ein Automat die beabsichtigte Sprache erkennt und nicht nur das gerade ausprobierte Wort. Ein Klick auf ein Ergebnis übernimmt das Wort wieder in die Einzelsimulation, um den Ablauf gezielt nachzuvollziehen.

Ein Automat lässt sich außerdem aus einer rechtslinearen Grammatik erzeugen, also aus den Regeln, nach denen die Wörter der Sprache gebildet werden. Jedes Nichtterminal wird dabei zu einem Zustand, und eine Regel der Form `A -> aB` wird zum Übergang von `A` nach `B` beim Lesen von `a`. Regeln der Form `A -> a` führen in einen zusätzlichen akzeptierenden Zustand, Regeln, die auf das leere Wort abbilden, machen den jeweiligen Zustand selbst akzeptierend. Für die Eingabe werden zwei Schreibweisen akzeptiert, eine knappe mit einer Produktion pro Zeile und eine strukturierte mit Kopfzeile. Das Format wird automatisch erkannt und angezeigt,

sodass zuvor keine Syntax ausgewählt werden muss. Die Aufteilung einer Alternative in Terminal und Nichtterminal erfolgt in zwei Durchläufen: Erst werden alle definierten Nichtterminale gesammelt, dann wird zerlegt. Das ist notwendig, weil Terminale und Nichtterminale denselben Namen tragen dürfen und sich `A -> AA` nicht aus den Zeichen allein auflösen lässt.

Beispiele und Übungen sind bewusst getrennt. Ein Beispiel ist ein fertiger Automat mit passendem Eingabewort und wird unverändert in die Simulation geladen; es dient dem Nachvollziehen eines vorhandenen Aufbaus. Eine Übung beginnt dagegen mit einem leeren Automaten und bringt Aufgabenstellung, Schwierigkeitsgrad, Referenzlösung und automatische Kontrolle mit.

Innerhalb beider Bereiche sind die Einträge danach gruppiert, was gegeben und was gesucht ist. Die Übungen zerfallen in zwei Kategorien: Bei *Sprache → Automat* ist die gesuchte Wortmenge umgangssprachlich beschrieben, bei *Grammatik → Automat* ist stattdessen eine rechtslineare Grammatik vorgegeben. Beide benutzen dieselbe Aufgabenmechanik und dieselbe Kontrolle; die zweite Kategorie trägt lediglich zusätzlich die Grammatik als Vorgabe. Die Beispiele sind entsprechend in deterministische Automaten, nichtdeterministische Automaten und die Umwandlung aus einer Grammatik unterteilt. Ohne diese Gliederung stünden Aufgaben mit unterschiedlicher Ausgangslage nebeneinander, und die Grammatikaufgaben wären allein an ihrem zusätzlichen Grammatikfeld zu erkennen.

Die Kontrolle der Übungen vergleicht nicht die Struktur zweier Automaten, sondern deren Verhalten. Sie prüft alle Wörter über dem gemeinsamen Alphabet bis zu einer festgelegten Länge; für jedes muss der Automat der Studierenden dieselbe Entscheidung treffen wie die Referenzlösung. Ein Strukturvergleich würde korrekte Lösungen ablehnen, die lediglich anders aufgebaut sind. Weicht eine Lösung ab, wird das erste unterscheidende Wort als Gegenbeispiel ausgegeben statt nur einer allgemeinen Fehlermeldung. Die begrenzte Wortmenge ist dabei bewusst eine didaktische Rückmeldung und kein formaler Äquivalenznachweis. Strukturelle Mängel wie ungültige Übergänge werden vorrangig gemeldet, da sie meist die eigentliche Ursache sind und eine Auflistung fehlgeschlagener Testwörter in diesem Fall in die Irre führen würde.

Der Bearbeitungsstand wird je Übung gespeichert und beim erneuten Öffnen wiederhergestellt. Ohne diese Trennung hätte das Wechseln zwischen zwei Übungen die jeweils andere Bearbeitung überschrieben.

4.3 Reguläre Sprachen (Kristian Popov)

Das Modul *Reguläre Sprachen* unterstützt die Arbeit mit regulären Sprachen und rechtslinearen Grammatiken. Es bietet drei miteinander verknüpfte Funktionen: Die Aufzählung der von einer Grammatik erzeugten Wörter, die Erzeugung einer passenden Grammatik aus einer Sprachbeschreibung sowie die Prüfung einzelner Wörter. Sprachbeschreibungen können entweder als Aufzählung oder in Mengenschreibweise eingegeben werden. Bei der Aufzählung einer Sprache werden die Wörter nach ihrer Länge sortiert. Die Ausgabe ist dabei auf eine maximale Wortlänge und Wortanzahl begrenzt. Bei der Wortprüfung zeigt das Modul neben der Annahme oder Ablehnung auch den Grund für ein abweichendes Ergebnis an, beispielsweise einen nicht möglichen Übergang. Beide Umwandlungsrichtungen sind zusätzlich als Übungen mit automatischer Rückmeldung verfügbar.

4.3.1 Entscheidungen in der Architektur (Kristian Popov)

Der Simulator besteht aus einer koordinierenden Komponente und fachlich getrennten Bausteinen. Die Komponente `RLSimulator` verwaltet die drei Arbeitsmodi, den Wechsel zwischen ihnen und den Einstieg in Übungen. Sie enthält selbst keine formale Logik. Diese liegt vollständig im Verzeichnis `domain` und ist damit unabhängig von React testbar.

Die fachlichen Bausteine sind wie folgt aufgeteilt:

Grammatikverarbeitung
Der Grammatikparser und die Validierung lesen die Eingabe ein, prüfen die Struktur einer rechtslinearen Grammatik und überführen sie in gemeinsame Datentypen.

Sprachbeschreibung
Ein eigener Parser verarbeitet Aufzählungen und Mengenschreibweisen. Er liefert eine strukturierte Beschreibung aus Faktoren, Exponenten und Nebenbedingungen, die anschließend in eine Grammatik umgewandelt werden kann.

Umwandlung und Vereinfachung
Die Umwandlung erzeugt aus einer Sprachbeschreibung eine strikt rechtslineare Grammatik. Ergänzende Funktionen fassen gleichwertige Nichtterminale zusammen und formatieren das Ergebnis für die Ausgabe.

Sprachauswertung
Die Aufzählung erzeugt Wörter bis zu den festgelegten Grenzen; die Worterkennung prüft ein einzelnes Wort gegen eine Grammatik. Die Übungsprüfung baut auf denselben Funktionen auf und ergänzt sie um die Auswertung der vorgegebenen Testfälle.

Wiederverwendet werden der Grammatikparser, die Grammatikvalidierung und die gemeinsamen Datentypen. Eine eigene Ableitungslogik direkt auf den Produktionen hätte zwei Implementierungen erfordert, die dasselbe Verhalten abbilden müssen. Mit der gemeinsamen Grundlage stimmen Aufzählung, Worterkennung und Übungsprüfung stets überein.

Für die Sprachbeschreibung habe ich einen eigenen Parser geschrieben, der zwei Formen versteht:

Aufzählung
listet die Wörter unmittelbar auf:
`L = {"010", "0010"}`

Mengenschreibweise
beschreibt ein Muster mit Variablen und Nebenbedingungen:
`L = {a^n c b^m | n, m > 0}`

Endet eine Aufzählung mit einer Fortsetzungsmarkierung, versucht der Parser, daraus ein wiederkehrendes Muster abzuleiten, statt die Wörter nur wörtlich zu übernehmen. So lässt sich eine unendliche Sprache angeben, ohne die Mengenschreibweise schon beherrschen zu müssen.

Bestimmte Eingaben werden bewusst abgelehnt, auch wenn sie sich formal beschreiben lassen. Verwendet dieselbe Variable mehrere Faktoren, wie in `a^n b^n`, koppelt das deren Längen aneinander. Der Musterfall `a^n b^n` ist nicht regulär; die unterstützte Sprachbeschreibung lässt solche gekoppelten Längenbedingungen deshalb generell nicht zu und weist sie mit einer eigenen Meldung zurück. Obere Schranken werden ebenfalls nicht unterstützt, obwohl sie endliche und damit reguläre Sprachen beschreiben. Sie würden die Eingabesprache unnötig erweitern und lassen sich klarer als Aufzählung angeben. Diese Fälle ausdrücklich zu erklären, ist lehrreicher, als sie stillschweigend zu verwerfen.

Die erzeugte Grammatik ist strikt rechtslinear, es entstehen also ausschließlich Regeln der Form `A -> aB` und `A -> a` sowie die Regel $A \to \varepsilon$. Eine frühere Fassung gab eine kompaktere Form wie `S -> A c B` aus. Diese Schreibweise ist zwar kürzer, aber eine allgemeine kontextfreie Regel und verlässt damit die Typ-3-Grammatiken, um die es in diesem Modul gerade geht.

Um trotzdem eine lesbare Ausgabe zu bieten, gibt es zwei Sichten auf dasselbe Ergebnis statt

zweier Formalismen: Die detaillierte Sicht zeigt ein Nichtterminal je Präfix, die vereinfachte Sicht führt Nichtterminale zusammen, die dieselben Wörter ableiten. Bei der vereinfachten Sicht werden äquivalente Teilbäume zusammengeführt. Sie bleibt rechtslinear und kann die Grammatik nur verkleinern, nie ihre Sprache verändern.

Alle Stellen, die eine rechtslineare Grammatik ausgeben, benutzen dieselbe Namensvergabe und Ausgabefunktion. Zuvor lag diese Logik in drei leicht unterschiedlichen Kopien vor. In einer davon verbrauchte ein weggelassener Endzustand einen Buchstaben, sodass in der Ausgabe eine Lücke entstand und etwa `E` übersprungen wurde. Eine gemeinsame Funktion schließt solche Abweichungen aus und stellt sicher, dass eine exportierte Grammatik nach dem Import dieselben Namen trägt.

4.3.2 Entscheidungen im UI (Kristian Popov)

Die drei Arbeitsweisen sind über eine Modulauswahl erreichbar. Ihre Komponenten bleiben im Hintergrund erhalten und werden nur ein- oder ausgeblendet. Dadurch bleiben Eingaben beim Wechsel zwischen den Modi bestehen. Würden die Modi jeweils neu aufgebaut, ginge die begonnene Arbeit verloren und die Ansicht spränge auf ihr Standardbeispiel zurück.

Ist eine Übung geöffnet, ersetzt eine geführte Arbeitsfläche den freien Umwandler. Vorgabe und Aufgabe stehen dabei nebeneinander: In der einen Richtung ist die Grammatik gegeben und die Sprache gesucht, in der anderen umgekehrt. Ein Moduswechsel während einer offenen Übung würde den aktuellen Bearbeitungsstand verwerfen. Deshalb wird vorher eine Bestätigung verlangt.

Fehlermeldungen erscheinen unmittelbar unter dem jeweiligen Eingabefeld, aber erst, nachdem überhaupt etwas eingegeben wurde. Ein leeres Feld ist kein Fehler, sondern der Ausgangszustand. Zusätzlich lässt sich die unterstützte Syntax direkt neben dem Eingabefeld aufklappen, damit die Schreibweise nicht erst gesucht werden muss.

4.3.3 Andere Entscheidungen (Kristian Popov)

Die Kontrolle der Übungen vergleicht keine Texte. Für die Richtung *Sprache*→ *Grammatik* sind zu jeder Aufgabe Wörter hinterlegt, die erzeugt werden müssen, und Wörter, die nicht erzeugt werden dürfen. Geprüft wird, ob die eingegebene Grammatik alle geforderten Wörter ableitet und alle ausgeschlossenen ablehnt. Ein Textvergleich mit einer Musterlösung wäre hier unbrauchbar, da dieselbe Sprache von vielen unterschiedlichen Grammatiken erzeugt werden kann und die Sprachen zum Teil unendlich sind. Die Rückmeldung nennt daher falsch erzeugte und fehlende Wörter statt nur einer Ja-Nein-Aussage. Da nur vorbereitete Beispiele geprüft werden, dient auch diese Kontrolle der gezielten Rückmeldung und ist kein allgemeiner Beweis für die Gleichheit zweier Sprachen.

Für die Richtung *Grammatik* → *Sprache* verwendet die Kontrolle dieselben Grenzen für Wortlänge und Wortanzahl wie die Schaltfläche zum Anzeigen der Lösung. Andernfalls könnte eine angezeigte Lösung die anschließende Prüfung nicht bestehen.

Beim Import und Export unterscheiden sich die beiden Formate bewusst. Eine Grammatik wird strukturiert kodiert, also in Nichtterminale, Terminale und Produktionen zerlegt und bitweise gepackt. Eine Sprachbeschreibung wird dagegen als Text übernommen, weil sie vom Benutzer selbst formuliert ist und ihre Schreibweise erhalten bleiben soll. Beide Seiten prüfen beim Kodieren und beim Dekodieren, dass der Inhalt gültig ist.

Jeder Modus speichert seinen Stand in einem eigenen Bereich der lokalen Ablage. Ohne diese Trennung hätten sich die Grammatik des Aufzählungsmodus und die des Worterkenners

gegenseitig überschrieben, da beide auf demselben Eingabetyp arbeiten.

Die Aufgabenstellungen selbst sind in einer sprachneutralen Notation hinterlegt und nicht in den Übersetzungsdateien. Eine Produktion wie `START = "1", A` ist in beiden Oberflächensprachen identisch; eine Übersetzung würde hier nur zusätzliche Fehlerquellen schaffen.

4.4 Zahlensysteme (Author)

4.4.1 Entscheidungen in der Architektur (Author)

4.4.2 Entscheidungen im UI (Author)

4.4.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.5 Moore & Mealy Automaten (Author)

4.5.1 Entscheidungen in der Architektur (Author)

4.5.2 Entscheidungen im UI (Author)

4.5.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.6 Disjunktive Normalform (Author)

4.6.1 Entscheidungen in der Architektur (Author)

4.6.2 Entscheidungen im UI (Author)

4.6.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.7 Encoding (Author)

4.7.1 Entscheidungen in der Architektur (Author)

4.7.2 Entscheidungen im UI (Author)

4.7.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.8 Java (Author)

4.8.1 Entscheidungen in der Architektur (Author)

4.8.2 Entscheidungen im UI (Author)

4.8.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Author)

CI/CD-Pipeline (GitLab Runner auf Raspberry Pi), automatisiertes Linting, Formatierung (Prettier), Unit-Tests (Vitest) und E2E-Tests (Playwright)

5.2 Abgleich mit dem Lastenheft (Author)

Wurden die Qualitätsziele (Performance, Usability, Barrierefreiheit durch Dark Mode und i18n) erreicht?

5.3 Lessons Learned (Author)

Herausforderungen in der Team-Kollaboration (Git-Workflow, Merge-Requests, geteilte Architektur) und technische Hürden

5.4 Design-Iterationen (Author)

Welche Entscheidungen mussten im Prozess revidiert werden?

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Author)

Sieben interaktive Fachmodule in zwei Lernbereichen

6.2 Perspektiven (Author)

Gamification (Badges/Punkte), LMS-Anbindung (Moodle/ILIAS) und Mobile Learning (responsive Optimierung)

Anhang

Abkürzungsverzeichnis

LSP Language Server Protocol; Protokoll zwischen Editor und Sprachserver für Codenavigation, Typinformationen und Refactoring

MCP Model Context Protocol; Offener Standard, über den Sprachmodelle externe Werkzeuge und Datenquellen anbinden

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [3]

Literatur

[1] S. Shamsi. „Graphify - Multi-modal Knowledge Graph Builder," besucht am 31. Juli 2026. Adresse: `https://graphify.net/`

[2] Sakana AI. „Sakana Fugu - Multi-agent System as A Model," besucht am 31. Juli 2026. Adresse: `https://sakana.ai/fugu/`

[3] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt," besucht am 14. Juni 2026. Adresse: `https://thws.de`